Scientific Computing: Exercises

Thorsten Koch

Applied Algorithmic Intelligence Methods department at ZIB

Chair of *Software and Algorithms for Discrete Optimization* at TU Berlin

L05

Answers

Exercise 5: Written part

1. What are the differences between the data structures Tree and Graph?
   Explain using the mathematical definitions of tree and graph.
   A tree is connected and cycle free Graph.
2. What is the maximum number of edges that a simple disconnected undirected graph with n vertices can have? Provide a proof to your answer.

$$\frac{(n-1)(n-2)}{2}$$

3. There are 23 routers in university building.
   Is it possible to connect them with cables such that each router is connected to exactly 5 others?
   Explain the reason.
   No. Because the sum of the degrees of all nodes in a graph has to be even.
4. In graph theory, a bipartite graph is a graph whose vertices can be divided into two disjoint and independent sets U and V, where every edge connects a vertex in U to one in V .
   Prove that the size of a bipartite graph of order (number of vertices) n is at most $\frac{n^2}{4}$.
   Each $u \in U$ can have at most $|V|$ edges, i.e. $|E| = |V| \cdot |U|$. Assuming $|U| = |V| = \frac{n}{2} \Longrightarrow \left(\frac{n}{2}\right)^2 = \frac{n^2}{4}$

1. What are the differences between the data structures Tree and Graph?
   Explain using the mathematical definitions of tree and graph.

Definition:

- Graph: A graph is a collection of nodes (vertices) and edges that connect pairs of nodes.
  Graphs can be directed or undirected, and they may have cycles.
- Tree: A tree is a specific type of graph that is acyclic (no cycles) and connected.
  It has a hierarchy with a root node, and each node (except the root) has a parent and zero or more child nodes.

Acyclic vs. Cyclic:

- Graphs: Graphs can have cycles (i.e., a sequence of edges that starts and ends at the same node).
- Trees: Trees are acyclic, meaning there are no cycles or loops in the structure.

Root and Hierarchical Structure:

- Graphs: Graphs typically do not have a root node, and nodes may not have a hierarchical relationship.
- Trees: Trees have a root node from which all other nodes are descended. Nodes have parent-child relationships, creating a hierarchical structure.

Connectivity:

- Graphs: Graphs can be disconnected, meaning there may be isolated groups of nodes with no connection between them.
- Trees: Trees are always connected, and there is a unique path between any pair of nodes.

Applications:

- Graphs: Used to model relationships, networks, dependencies, and various real-world scenarios where entities are connected.
- Trees: Often used in hierarchical structures like file systems, organizational charts, and representing hierarchical relationships in data.

2. What is the maximum number of edges that a simple disconnected undirected graph with n vertices can have? Provide a proof to your answer.

$G = (V, E)$ is simple (no self loops, exactly one edge between two vertices), undirected, disconnected, with $|V| = n$.

We need to find $\max(|E|)$.

Hypothesis:

Due to disconnectedness of G, $\max(|E|)$ is same as the maximum number of edges in a connected graph with $n - 1$ vertices which is the number of distinct pairs our of $n - 1$ vertices:

$$\max(|E|) = \binom{n-1}{2} = \frac{(n-1)(n-1-1)}{2} = \frac{(n-1)(n-2)}{2} = \frac{n^2 - 3n + 2}{2}$$

Proof:

✓ Proof by induction

Alternative solution:

To have the maximum number of edges, G must have at most two disconnected parts $G_1 = (V_1, E_1)$, and $G_2 = (V_2, E_2)$ such that $G = G_1 \cup G_2$, and G_1 and G_2 are connected. Therefore, V_1 and V_2 are disjoint sets such that $|V_1| = a$ and $|V_2| = n - a$.

$$\max(|E|) = \max_{1 \le a < n} \left(\binom{a}{2} + \binom{n-a}{2} \right)$$

Take first and second derivates of the resulting function to show that the a value maximizing the function is 1.

Therefore, $\max(|E|) = \frac{(n-1)(n-2)}{2}$

3. There are 23 routers in university building.
   Is it possible to connect them with cables such that each router is connected to exactly 5 others?
   Explain the reason.

No.

The router network is an undirected graph with n=23 nodes.

Degree of a node i, $\deg(i)$, is the number of edges adjacent to the node i.

In an undirected graph, an edge is adjacent to two nodes, so every additional edge adds two to the total degree of nodes, one for each adjacent node's degree. Hence,

$$\sum_{i \in V} \deg(i) = 2|E| \text{ (**ref**. *handshaking lemma or degree sum formula*)}$$

In the question, 23x5=115 is the total degree of nodes, if each router of the 23 routers are connected to exactly 5 other nodes. As 115 is an odd number, so it is not possible following the degree sum formula.

4. In graph theory, a bipartite graph is a graph whose vertices can be divided into two disjoint and independent sets U and V, where every edge connects a vertex in U to one in V .

   Prove that the size of a bipartite graph of order (number of vertices) n is at most $\frac{n^2}{4}$.

Let $|U| = p$, then $|V| = n - p$. The number of edges between U and V is $|E| = p(n-p)$:

$$\max(|E|) = \max_{p \in Z^+}(p(n-p)) \rightarrow p^* = \frac{n}{2};\ \max(|E|) = \frac{n^2}{4} \text{ if n is even;}$$

$$p^* = \frac{n-1}{2};\ \max(|E|) = \frac{n^2-1}{4} \text{ if n is odd.}$$

5. How does the Kronecker product of a complete graph with itself look like?

The ***Kronecker (tensor) product of two matrices A and B*** is: $A \otimes B = \begin{bmatrix} a_{11}B & \cdots & a_{1m}B \\ a_{21}B & \cdots & a_{2m}B \\ \vdots & \ddots & \vdots \\ a_{n1} & \cdots & a_{nm}B \end{bmatrix}$

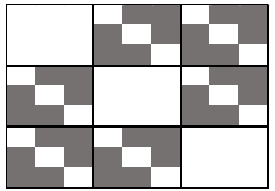
The ***Kronecker product of two graphs*** is the Kroenecker product of their adjacency matrices.

Let the graph be $G = (V, E)$ where G is complete. Adjacency matric of G, $A(G)$ is an nxn matrix, with ones except for the diagonal,

which is 0: $a_{ij} = \begin{cases} 0, & \text{if } i = j \\ 1, & \text{otherwise} \end{cases} \Rightarrow \mathrm{G} = \begin{bmatrix} 0 & 1 & \cdots & 1 \\ 1 & 0 & \ddots & \vdots \\ \vdots & \ddots & \ddots & 1 \\ 1 & \dots & 1 & 0 \end{bmatrix}$

$n = 3$:

$A(G) = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}$, $A(G') =$

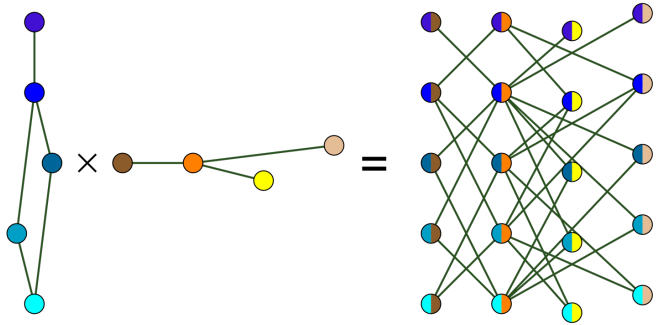


Let $G' = G \otimes G$ such that $G' = (V', E')$ Then, from definition, $A(G') = A(G) \otimes A(G)$

Therefore:

- $A(G')$ is an $n^2 \times n^2$ matrix $\Rightarrow$ G' has n^2 nodes: $|V'| = n^2$
- $A(G')$ has a block-diagonal structure of nxn matrices, where the block in the diagonals are zero and other blocks are G

$\Rightarrow$ Hence, we can place the nodes of G' over an $n \times n$ grid so that the nodes of G' are adjacent to the nodes that are not in the same row and same column on the grid.



Graph

```rust
pub type NodeNo = u32;
pub type Weight = u32;

streckt Neighbor {
    head : NodeNo,
    dist : Weight,
}
impl Neighbor {
    fn new(head : NodeNo, dist : Weight) -> Self {
        Neighbor { head, dist, }
    }
}
struct Node {
    neighbors : Vec<Neighbor>,
}
impl Node {
    fn new() -> Self {
        Node { neighbors : Vec::new(), }
    }
    fn add_neighbor(
        &mut self, head : NodeNo, dist : Weight) {
        self.neighbors.push(Neighbor::new(head, dist))
    }
}
```

```rust
pub struct Graph {
name  : String,
    nodes : Vec<Node>,
}


impl Graph {
    pub const INVALID_NODE : NodeNo = NodeNo::MAX;
    pub const FAR_FAR_AWAY : Weight = Weight::MAX;

    pub fn new(name : &str) -> Self {
        Graph { name : name.to_string(),
            nodes : Vec::new(),
        }
    }


// Alternative:

pub struct Graph {
    begin : Vec<Node>, // size nodes + 1
    heads : Vec<EdgeNo>;
    dists : Vec<EdgeNo>;
}
```

```rust
/// Runs a breadth first search (BFS) starting from node start
/// and stores the depth of the nodes.
fn bfs(&self, start : NodeNo, depth : &mut [usize]) -> usize {
    debug_assert!((start as usize) < self.node_count(), "start node >= node count");
    let mut queue = VecDeque::new();
    let mut dmax  = 1;

    depth[start as usize] = dmax;
    queue.push_back(start);

    while !queue.is_empty() {
        let tail = queue.pop_front().unwrap();
        let d    = depth[tail as usize];

        for e in &self.nodes[tail as usize].edges {
            debug_assert!(depth[e.head as usize] <= depth[tail as usize] + 1);
            if depth[e.head as usize] == 0 {
                queue.push_back(e.head);
                debug_assert!(d == dmax || d == dmax - 1);
                dmax = d + 1;
                depth[e.head as usize] = dmax;
            }
        }
    }
    dmax - 1
}
```

Maybe better to set size

Connected components

```rust
/// Runs a breadth first search (BFS) starting from node start
/// and computes the depth of the resulting of tree.
pub fn bfs_depth(&self, start : NodeNo) -> usize {
    assert!((start as usize) < self.node_count(), "start node >= node count");
    let mut depth = vec!(0; self.node_count());

    self.bfs(start, &mut depth)
}

/// Find the number of connected components.
/// To do so, we run a BFS from every no visited node.
pub fn connected_components(&self) -> usize {
    let mut depth = vec!(0; self.node_count());
    let mut comps = 0;

    for n in 0..self.node_count() {
        if depth[n] == 0 {
            comps += 1;
            self.bfs(n as NodeNo, &mut depth);
        }
    }
    comps
}
```

Giving a certificate for the components

How could we give some additional information, that would make it easy to check whether the components we computed are correct?

1. To every node add the component number
2. To every node add a sequence number within the component.

We number the vertices within a component such that every vertex except for one has a lower numbered neighbor.

Such a numbering proves that every vertex within a component is connected to the vertex with the smallest vertex number.

To check this certificate, it suffices to check, for each node, that its labels are nonnegative, for each edge, that the endpoints have the same component number and distinct vertex numbers within the component, and to mark the endpoint with the larger number if it is not already marked, and for each component, that exactly one of its vertices is unmarked.

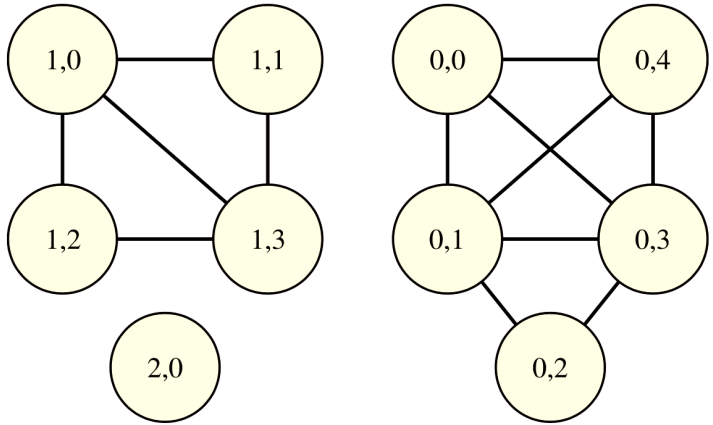


Fig. 3 – A graph with three connected components. The vertices are labeled with pairs (i, j). The first label is the number of the component to which the vertex belongs and the second label is the number within the component. Within a component, every vertex has a smaller numbered neighbor except for the vertex numbered zero.

Certifying algorithms; McConnella, Mehlhorn, Näherc, Schweitzerd
DOI:10.1016/j.cosrev.2010.09.009
An Introduction to Certifying Algorithms; Alkassar, Böhme, Mehlhorn, Rizkallah, Schweitzer. DOI:10.1524/itit.2011.0655

Details

```
let input          = BufReader::new(stream);
let mut g          = Graph::new();
let mut node_count = 0;
let mut edge_count = 0;

for (lineno, line) in input.lines().enumerate() {
    let data               = line.unwrap_or_else(|err| panic!("Line {} {err}", lineno + 1));
    let fields : Vec<&str> = data.split_whitespace().collect();

    // The first non comment line has the number of nodes and edges
    if fields.len() >= 2 && node_count == 0 && edge_count == 0 {
        node_count = fields[0].parse::<usize>().unwrap_or_else(|err| panic!("Line {} {err}, expected number of nodes", lineno + 1));
        edge_count = fields[1].parse::<usize>().unwrap_or_else(|err| panic!("Line {} {err}, expected number of edges", lineno + 1));
        // ...
        continue;
    }
    // Other lines have format: Tail-Node-No Head-Node-No Weight
    if fields.len() >= 3 && node_count > 0 && edge_count > 0 {
        let tail : NodeNo = fields[0].parse().unwrap_or_else(|err| panic!("Line {} {err}, expected node no", lineno + 1));
        let head : NodeNo = fields[1].parse().unwrap_or_else(|err| panic!("Line {} {err}, expected node no",lineno + 1));
        edge_count -= 1;
        g.add_edge(head - 1, tail - 1, dist);
        continue;
    }
    panic!("Line {} syntax error", lineno + 1);
}
if edge_count != 0 {
    panic!("End of file: {} edges missing", edge_count);
}
```

What happens if we just write `.parse()` and the file would read like this:

```
4 -6
1 2 17
2 3 14
```

Same effect

`node_count`/`edge_count` would be read as i32, -6 would be a valid value, and we would get a syntax error at line 2. Why?

Automated Tests: cargo test

```rust
#[test]
#[should_panic(expected = "Line 1 invalid digit found in string, expected number of edges")]
fn check6() {
    let s = b"4 -6\n1 2 8\n1 4 1\n1 3 3\n2 3 5\n2 4 4\n3 4 2 \n";

    // it is possible to read a graph from a string using Cursor
    let mut input: Box<dyn Read> = Box::new(Cursor::new(s));
    let _ = Graph::read_from_stream(&mut input);
}
```

```
cargo test (in directory: /home/bzfkocht/project/TU-SciComp-Code/13-graph/rust/graph/src)
   Compiling graph v0.1.0 (/home/bzfkocht/project/TU-SciComp-Code/13-graph/rust/graph)
    Finished test [unoptimized + debuginfo] target(s) in 0.70s
     Running unittests src/main.rs (/home/bzfkocht/project/TU-SciComp-Code/13-graph/rust/graph/t
running 6 tests
test check2 ... ok
test check1 ... ok
test check4 - should panic ... ok
test check5 - should panic ... ok
test check6 - should panic ... ok
test check3 ... ok
test result: ok. 6 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
Compilation finished successfully.
```

You can also test for errors.

Ideally, there is a test case for every possible error in the program.

- cargo is a build system.
- It knows about dependencies between files (how?).
- And it knows how to generate one file type from another.
- It automatically just executes the minimum necessary commands to update everything.

```
cargo build
cargo run
cargo clippy
cargo test
cargo bench
cargo doc --open
cargo run --release
cargo-fmt
```

```
backyard
├── Cargo.lock
├── Cargo.toml
└── src
    ├── garden.rs
    └── main.rs
```

Use if main.rs does not do much other than call the code in garden.rs

```
backyard
├── Cargo.lock
├── Cargo.toml
└── src
    └── bin
        └── vegetables.rs
```

Advantage: Not all your files are named main.rs

L06

Network algorithms

This Exercise is about finding the minimum spanning tree (MST) of a connected undirected graph and finding the length of the shortest path (SP) between two vertices in a graph:

Write a Rust program to compute the

1. weight of the minimum spanning tree of a graph
2. length of shortest path between two vertices in a graph

Use the graphs on ISIS and here to test: https://www.zib.de/userpage/koch/SP/data

If called as:

```
$ cargo run b01.gph 2 45
```

The output should be something like:

```
b01 MST= 238 SP= 9 Path: 2 19 18 28 45 Time: 145 ms
```

`b01.gph` is the file with the graph (same format as last week).
2 and 45 are the start and end vertices for which the length of the shortest path should be computed.
The numbers after Path: are the vertices along the shortest path between the two vertices.
The time is the total time need for reading, and computing the MST and SP.

Assignment 6: Written part

1. Prove why the invariant of the Dijkstra's algorithm guarantees that the solution is indeed a shortest path.
2. Dijkstra's algorithm is known for solving the shortest path problem, but it requires that the edge weights be non-negative. Give an explanation or an example to show that the Dijkstra's algorithm does not always find the shortest path in case of negative edge weights. List at least one algorithm to find shortest path in case of negative weights.
3. Let G be a graph and let l be a non-negative length function on the edges. Define a function f on the vertex pairs by letting $f(u, v) =$ the length of a shortest $u - v$ path in G with respect to l.
   Show that f (which is the "distance function") satisfies the triangle inequality, that is, for all triples x, y, z of vertices it holds that $f(x, y) \leq f(x, z) + f(z, y)$.
4. Provide an algorithm to decide whether it is possible to construct a minimum spanning tree (MST) of a connected graph containing an arbitrarily chosen edge without finding the MST of the graph explicitly as a part of the algorithm.
5. Kruskal's algorithm and Prim's algorithm are known to find the MST of a connected weighted graph. By using each of these algorithms, is it possible to find a spanning forest of minimum weight in a disconnected weighted graph? If not, give the required modification.
6. Find an $s - t$ flow of maximum value and an $s - t$ cut of minimum capacity in the following graph: